CONTROL STRUCTURES

- **if, else**
- **switch, case, default, break**
- **for**
- **while**
- **do, while**

CONTROL STRUCTURES: IF AND IF, ELSE

Example of an **if** statement

```
public class Test
{
   public int variable =20;

   public Test ()
   {
      if ( variable == 20)
      {
         System.out.println ("The variable is indeed equal to 20");
      }
   }
}
```


Example of an **if, else** statement

```
public class Test
{
   public int variable = 15;

   public Test ()
   {
      if ( variable == 20)
      {
         System.out.println ("The variable is indeed equal to 20");
      }
      else
      {
         System.out.println ("The variable is not equal to 20");
      }
   }
}
```

CONTROL STRUCTURES: WHILE

- **While**: Indefinite Loops
- We want to repeat one or more instructions an indefinite number of times: we repeat the instruction or block of instructions as long as a certain condition remains true.
- In Java, we have a basic while loop: while ***(condition) {block of instructions}***
  - The instructions within the block are repeated as long as the condition remains true.
  - We never enter the loop if the condition is false from the start.

CONTROL STRUCTURES: DO… WHILE

- **Another type of loop using the "while" loop:**
  - do ***{block of instructions}*** while ***(condition)***
  - **The instructions within the block are repeated as long as the condition remains true.**
  - **The loop always enters the condition at least once.**
  - **The condition is tested at the end of the loop.**

CONTROL STRUCTURES: FOR

- **for**: Determinate Loops
- We want to repeat one or more instructions a specific number of times: we repeat the instruction or block of instructions for a certain number of steps.
- The for loop:

```
for (int i=1; i<=10; i++)
system.out.println(i) //displays the numbers from 1 to 10
```


CONTROL STRUCTURES: SWITCH

- **Switch**: Multiple Selections
- Using if/else can be cumbersome when handling multiple selections and alternatives.
- Java offers a solution for this: the **switch/case** statement.
- The parentheses following the keyword `switch` indicate an expression whose value is tested successively by each case.
- When the tested expression matches one of the values following a case, the list of instructions that follows it is executed.
  - The keyword `***break***` indicates the exit from the conditional structure.
  - The keyword `***default***` precedes the list of instructions that will be executed if the expression is never equal to one of the values.

The switch statement only works on simple types (int, short, char or byte)

```
switch (Variable) {
case Value1 :
                    List of instructions
                    break;
case Value2 :
                     List of instructions
                     break;
case Values... :
                    List of instructions
                    break;
default:
                    List of instructions
                    break;
}
```

CONTROL STRUCTURES: EXAMPLE

if, else if

```
public class Test
{
   public int variable = 15;

   public Test()
   {
      if(variable == 20)
      {
         System.out.println("The variable is indeed equal to 20");
      }
      else if(variable == 15)
      {
         System.out.println("The variable is not equal to 20 and is equal to 15");
      }
      else if(variable == 10)
      {
         System.out.println("The variable is neither equal to 20 nor to 15 and is equal to 10");
      }
else
      {
         System.out.println("The variable is neither equal to 20 nor to 15 nor to 10");
      }
   }
}
```

switch

```
public class Test
{
   public int variable = 15;

   public Test()
   {
      switch(variable)
      {
         case 20 :
            System.out.println(" The variable is indeed equal to 20");
         break;
         case 15 :
            System.out.println(" The variable is not equal to 20 and is equal to 15 ");
         break;
         case 10 :
            System.out.println(" The variable is neither equal to 20 nor to 15 and is equal
to 10 ");
         break;
         default:
            System.out.println("The variable is neither equal to 20 nor to 15 nor to 10");
break;
      }
   }
}
```

THE CLASS *SCANNER*

- Defined starting with JDK 1.5.
- Facilitates reading from a stream.
- Particularly useful for reading data from the keyboard in a console application (DOS/terminal).
- Defined in the java.util package.
- To use it, you must import it:
  - Write the following as the first instruction in the source file:
  - *import java.util.*;*
- It includes several methods, such as the *next()* method.
- The *next()* method blocks execution until the data is read and returns it as a string.
- It also has the *nextXXX()* methods where *XXX* represents a primitive type, for example: nextLine(), nextInt(), etc.

THE CLASS *SCANNER*

- Example:

```
import java.util.*;
public class ScannerExample {
public static void main(String args[]){
        Scanner in = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = in.nextLine();
        System.out.println("Name is: " + name);
        in.close();
        }
}
```


EXERCISE 3 (FOLLOWING)

- Change the previous program so that a and b are variables read from the keyboard.

```java
public class SwapNumbersInput {
    public static void main(String[] args) {
        // Create a Scanner object to read input
        Scanner reader = new Scanner(System.in);
        int temp;

        // Prompt the user for input
        System.out.print("Enter value for a: ");
        int a = reader.nextInt();

        System.out.print("Enter value for b: ");
        int b = reader.nextInt();

        System.out.println("\nBefore swapping: a = " + a + ", b = " + b);

        // Swapping logic
        temp = a;
        a = b;
        b = temp;

        System.out.println("After swapping: a = " + a + ", b = " + b);

        // Close the scanner to free resources
        reader.close();
    }
}
```

EXERCISE 4

- Write a program called Factorial that allows the user to enter a natural number N and then displays its factorial on the screen.

```java
import java.util.Scanner;

public class Factorial {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // 1. Prompt the user for a natural number (positive integer)
        System.out.print("Enter a natural number N: ");
        int n = input.nextInt();

        // 2. Validate that the number is not negative
        if (n < 0) {
            System.out.println("Error: Please enter a non-negative integer.");
        } else {
            long result = 1; // Use 'long' to handle larger results

            // 3. Calculate factorial using a loop
            for (int i = 1; i <= n; i++) {
                result *= i; // result = result * i
            }

            // 4. Display the output
            System.out.println("The factorial of " + n + " (" + n + "!) is: " + result);
        }

        input.close();
    }
}
```